[LoLShinSouls, MapTool]

[DX11기반 렌더러 제작 및 팀게임 포트폴리오]



PERIOD	약 2달 (2023.03 ~ 2023.05)
ROLE	팀프로젝트 (3인, 팀장), Map/MapTool, UI
STACK	C++11, DX11/HLSL, FMODSDK, FBXSDK
SUMMARY	DXAPI 기반 렌더링 프레임워크 및 협업과정 을 통한 3D액션게임제작
GitHub	https://github.com/ckdlscjs/ToolEngine, https://github.com/ckdlscjs/TeamProject

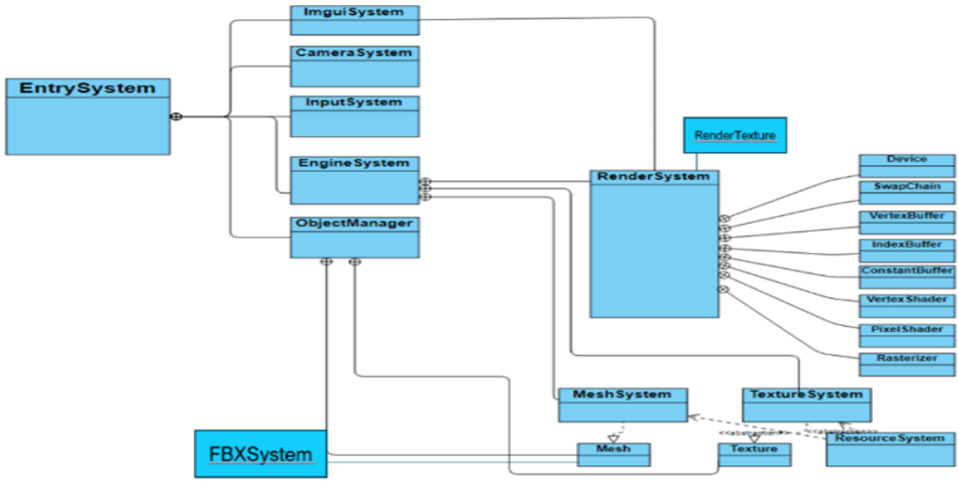
00 [프로젝트 개요]

문제인식 및 개발배경

본 프로젝트는 DirectX 11과 C++11을 기반으로 하여 구성된 팀원과 역할을 분담해 각자 설계한 프레임워크를 기반으로 렌더링 및 게임 클라이언트 개발을 위해 구성을 나누어 기능개발을 진행하였다

프로젝트 목표

- **개인 렌더러 & 맵 에디터 (MapTool)**
  - 지형 높낮이를 실시간 코사인 가중치로 깎는 Sculpting 브러시, 알파 가중치 변조 4중 Splatting 브러시 및 대량의 식생/에셋 배치 최적화 모듈을 직접 구현하여 즉각적인 3D 씬을 구성함
  - 월드에 배치된 충돌 상자(OBB/AABB Collider), 몬스터/플레이어 스폰 포인트, 시네마 카메라 무빙 궤적 노드 등을 오차 없이 파일로 직렬화(Save/Load) 출력 및 관리하는 파이프라인을 구축함.



- **3D 액션 RPG 게임 클라이언트 (LoLShinSouls)**
  - 리그 오브 레전드(LoL)의 리소스를 활용, 레퍼런스가 되는 액션 게임방식을 (보스 레이드 AI 패턴, 이동 및 회피, 타격 경직 및 HP 게이지바 상태 연동 등)을 D3D11 런타임 상에서 표현
  - 맵 에디터에서 직렬화 출력된 월드 씬 파일을 실시간 역직렬화 복원하고, 장애물 시야 가림을 차단하는 SpringArm 3인칭 카메라 및 RTT 미니맵 가속 렌더 패스를 탑재하여 최종 플레이 성능을 실시간 60FPS 이상으로 안정적으로 방어함.

핵심 시스템 구성

기반구조	싱글턴 기반 코어 프레임워크 및 D3D11 래퍼클래스 구성, 셰이더 컴파일러 및 렌더링 라이프사이클 관리
에셋 파이프라인	FBX SDK 재귀 노드 파싱 및 std::wstring 경로 키 기반 캐싱, FbxObject 기하 버퍼 자원화
지형 편집	쿼드트리 기반 공간분할, 코사인 높이 변형(스컬프팅) 및 4중 텍스처 알파 가중치 실시간 CPU 변조 및 GPU 전송(스플래팅)
드로우콜 최적화	수천 개 식생의 월드 행렬을 InstanceBuffer에 수집하여 드로우콜 1회(DrawIndexedInstanced) 고속 렌더링
카메라 시스템	OBB 레이캐스트 충돌 회피(Lerp Lag 보정), n차 베지어 경로 곡선 보간 및 펄린 노이즈 쉐이크 카메라 연출
UI시스템	상속 기반 Interface 계층 트리 구조(m_pChildList) 상대 좌표 제어, InterfaceWork 비동기 연출 추상화, 미니맵 렌더링

01 [Render Framework & MapTool]

[1. 3D렌더링 프레임워크 및 에셋/직렬화 파이프라인]

문제인식 및 필요성

- **독자적 렌더링 프레임워크 설계**: DXAPI 제어 구조를 바닥부터 설계하여 월드 공간 내 배경(지형) 에셋 로딩, 파싱, 리소스 바인딩 및 UI 렌더링 파이프라인의 통합 관리 인프라 필요.

설계

- **공통 프레임워크 생명주기**: Engine, Render, Resource 등 핵심 시스템의 싱글톤 매니저 관리 하에 Init -> Update -> Render -> Release 흐름 구조화.
- **에셋 관리 채널**: 에셋 경로명을 키로 사용하는 Key-Value 맵 구조로 중복 파일 로딩을 차단하고, 가상 정의 에셋을 D3D11 버퍼/상태 객체에 1:1 매핑.
- **FBX SDK 파이프라인**: FBX SDK로 정점, 인덱스, 애니메이션 계층 노드를 재귀 순회 파싱하여 FBXFile 및 FBXObject 구조 구축 후 D3D11 자원화.
- **공간 분할(Quadtree) 연동**: 맵 에셋 및 충돌체를 쿼드트리 노드 계층 내에 적재하여 가시성 판정(Frustum Culling) 및 정밀 충돌 탐색 비용 최적화.
- **씬 데이터 파일 직렬화**: 출력연산자 오버로딩을 기동하여 지형 데이터 및 배치된 액터 정보(Transform, 에셋 명칭 등)를 구조화된 텍스트 형식으로 직렬화/역직렬화(Save/Load) 처리.

[2.지형편집 브러시(Sculpting/Splatting) 및 폴리지 인스턴싱]

문제인식 및 필요성

- **지형 연산 병목**: 대규모 지형 렌더링 시 정점 급증으로 드로우콜 부하가 발생하며, 마우스 편집 브러시 작동 시 모든 정점을 전수 순회하는 $O(N)$ 연산 병목 발생.
- **개별 드로우콜 오버헤드**: 대량의 풀, 나무 등 식생(Foliage) 에셋을 개별 드로우할 경우 API 드로우콜 한계 직면.

설계

- **쿼드트리 기반 공간 조형 (Sculpting)**:
  - 마우스 피킹 위치를 중심으로 쿼드트리 공간 분할 노드를 추적하여 반경 내의 리프 노드 및 대상 정점 인덱스 세그먼트만 고속 필터링(SelectVertexList).
  - 코사인 감쇠 공식을 사용해 브러시 반경 내의 높이값(Y축)을 변형하고 정점 법선 벡터를 재계산하여 음영을 실시간 갱신(UpdateVertexBuffer).
  - 배치 오브젝트들의 지형 밀착을 위해 객체 위치의 지형 삼각폴리곤 높이 평균값을 역산해 높이 동기화 처리.
- **4중 텍스처 스플래팅 (Splatting) 브러시**:
  - 지형용 R,G,B,A 채널 기반 마스크 텍스처를 동적 생성 및 픽셀 셰이더(PS)에 바인딩.
  - 마우스 드래그 시 마스크 텍스처의 알파 정보(가중치 0.0~1.0)를 UpdateSubresource로 실시간 변경 및 픽셀 셰이더 내 선형 보간(lerp) 기반 변조(Modulate) 처리로 부드러운 텍스처 혼합 렌더링 구현.
- **Foliage 인스턴싱 및 SkySphere**:
  - 식생 개체들을 일괄 렌더링하기 위해 InstanceDataBuffer에 개별 월드 행렬을 적재하여 단 1회의 드로우콜(DrawIndexedInstanced)로 처리.
  - 구체(Sphere) 생성 알고리즘 기반 정점 데이터로 SkySphere 기하 자원을 구축하고, 백버퍼 맨 뒤에 배치 렌더링하여 효율적인 배경 하늘 구성.

02 [LoLShinSouls 게임 클라이언트 개발]

[1. 3인칭 및 시네마 카메라]

문제인식 및 필요성

- 3D 캐릭터 중심의 뷰 행렬 구성 외에도 지형 지물 충돌에 따른 카메라 거리 조절(장애물 가림 방지) 및 부드러운 카메라 조작 기능 설계 필요.

- 보스 조우 컷씬 및 타격감 전달을 위한 셰이크 연출 등 시각적 몰입도 향상 인프라 구축 필요.

설계

- **3인칭 카메라 및 장애물 충돌 회피 (SpringArm)**:
  - 캐릭터 위치 기준의 구면 좌표계 카메라 뷰 세팅.
  - 카메라 방향으로 레이캐스팅(Ray to OBB)을 가동하여 지형지물 충돌 검출 시 스프링암 길이를 단축하고, 프레임 변화량(DeltaTime) 보간 연산으로 부드러운 시점 복귀 구현.
- **시네마틱 및 흔들림 연출 (Bezier Curve & Perlin Noise)**:
  - **카메라 경로 제어**: 맵 에디터에서 지정한 경로 노드 데이터를 직렬화 로드하고, n차 베지어 곡선 보간을 적용해 매끄러운 컷씬 카메라 이동 경로 설계.
  - **카메라 셰이크**: 난수 기반 오프셋 대신 1D/2D 펄린 노이즈 분포 샘플링 알고리즘을 도입해 진폭과 주파수 기반의 연속적이고 자연스러운 카메라 흔들림 연출 구현.

[2. UI 프레임워크]

문제인식 및 필요성

- 인게임 UI 개체(버튼, 쿨타임 타이머, 플로팅 텍스트 등)들의 개별 렌더링으로 인한 드로우콜 낭비를 방지하고, UI 상태 동기화 및 갱신 흐름의 체계적인 관리가 가능한 독립 프레임워크 레이어 필요.

설계

- **트리 구조 기반 UI 프레임워크**:
  - 공통 부모 Interface 클래스를 구현하고, 자식 자원 포인터를 소유하는 계층 트리 구조(Parent-Child)로 설계하여 화면 해상도 변화 시 부모 좌표 기준 상대 좌표 일괄 동기화.
  - 페이드인/아웃, 쿨타임 스케일 변화 기능을 비동기 작업 클래스 InterfaceWork로 추상화하여 런타임 프레임 동기화 갱신 자동화.
- **3D 공간 빌보드 (Billboard) UI**:
  - 월드 공간 내 2D 체력바 및 플로팅 데미지 출력을 위해, 뷰 행렬의 역행렬 방향 벡터를 실시간 연산하여 카메라 회전과 관계없이 항상 정면을 지향하는 빌보드 UI 매핑.
- **시계방향 스킬 쿨타임 페이드 최적화**:
  - CPU 단의 매 프레임 정점 변형 연산을 배제하고, 픽셀 셰이더(PS) 상에서 쿨타임 진행률 상수버퍼와 atan2 연산으로 중심각을 계산해 픽셀 컬러를 변조하는 픽셀 셰이더 연산 가속 처리.
- **미니맵 RTT 및 기하 셰이더 최적화**:
  - 탑뷰 시점 카메라 뷰를 렌더 타겟 텍스처(RTT)로 드로우하여 미니맵 UI 뷰포트 출력.
  - 지형 외의 가동 에셋들은 개별 정점 데이터를 파이프라인에 주입하지 않고, 기하 셰이더(GS) 내부에서 1개의 원시 좌표 정점만으로 박스 메시를 생성하는 기하 가속을 적용해 버퍼 전송 오버헤드 억제.

결과 및 개선여지

종합 활용 결과

- 맵툴 에디터 파이프라인과 게임 엔진 간 씬 데이터 직렬화 호환성을 확보하여 3D 지형, 식생 인스턴싱, 충돌 볼륨을 유기적으로 연동 개발 완료.
- 쿼드트리 공간 분할 및 절두체 컬링, 기하 셰이더 활용 렌더링 가속 등 시스템 전반의 최적화를 통해 실전 프레임 유지.

개선여지

- **충돌 검출 가속의 한계**: CPU 단 마우스 피킹 및 충돌 연산량이 늘어남에 따라 병목이 체감되므로, 해당 연산을 GPU 비동기 Compute Shader로 이관하는 최적화 구조 변경 필요.
- **자원 바인딩 비용 최소화**: 객체지향 기반 렌더러 루프 상의 D3D11 장치 상태 전환 오버헤드가 크므로, 정렬 키 기반 렌더패스 최적화 기법을 도입한 전면 개편 필요

개발과정 세부사항

렌더러 및 맵에디터 영상

게임 영상(LoLShinSouls)